

Controller – Melihat Pelamar

```
func GetJobApplicants(c *gin.Context) {  
    jobID := c.Param("id")  
    userID, ok := getUserID(c)  
  
    if !ok {  
        c.JSON(http.StatusUnauthorized, gin.H{"error": "Unauthorized"})  
        return  
    }  
  
    var job models.Job  
  
    if err := config.DB.First(&job, jobID).Error; err != nil {  
        c.JSON(http.StatusNotFound, gin.H{"error": "Job tidak ditemukan"})  
        return  
    }  
  
    if job.ClientID != userID {  
        c.JSON(http.StatusForbidden, gin.H{"error": "Tidak bisa lihat pelamar job orang lain"})  
        return  
    }  
  
    var apps []models.Application  
  
    err := config.DB.  
        Preload("Freelancer").  
        Where("job_id = ?", jobID).  
        Order("tanggal_daftar desc").  
        Find(&apps).Error  
  
    if err != nil {  
        c.JSON(http.StatusInternalServerError, gin.H{"error": "Gagal ambil pelamar"})  
        return  
    }  
  
    c.JSON(http.StatusOK, apps)  
}
```

Penjelasan :

“Function GetJobApplicants ini digunakan untuk menampilkan daftar pelamar pada suatu proyek milik client. Alurnya dimulai dengan mengambil jobID dari parameter URL, lalu sistem mengecek apakah user yang mengakses sudah login dengan mengambil userID. Setelah itu, sistem mencari data proyek berdasarkan jobID. Jika proyek tidak ditemukan, maka akan ditampilkan error. Selanjutnya dilakukan validasi untuk memastikan bahwa hanya client pemilik proyek yang bisa melihat daftar pelamar. Jika bukan pemiliknya, akses akan ditolak.

Jika semua validasi sudah lolos, sistem akan mengambil data pelamar dari database berdasarkan job_id. Pada bagian ini juga digunakan Preload untuk mengambil data freelancer yang melamar, sehingga informasi yang ditampilkan lebih lengkap. Data kemudian diurutkan berdasarkan tanggal pendaftaran terbaru. Terakhir, sistem akan mengembalikan hasilnya dalam bentuk JSON yang berisi daftar pelamar. Jadi, fungsi ini memastikan bahwa data pelamar hanya bisa diakses oleh pihak yang berhak dan ditampilkan secara lengkap kepada client.”

Controller – Mengelola Proyek - Delete

```
func DeleteJob(c *gin.Context) {  
    id := c.Param("id")  
    userID, ok := getUserID(c)  
    if !ok {  
        c.JSON(401, gin.H{"error": "Unauthorized"})  
        return  
    }  
    var job models.Job  
    if err := config.DB.First(&job, id).Error; err != nil {  
        c.JSON(404, gin.H{"error": "Job tidak ditemukan"})  
        return  
    }  
    if job.ClientID != userID {  
        c.JSON(403, gin.H{"error": "Bukan job milikmu"})  
        return  
    }  
    tx := config.DB.Begin()
```

```
if err := tx.Model(&job).Update("status", "dihapus").Error; err != nil {  
    tx.Rollback()  
    c.JSON(500, gin.H{"error": "Gagal menghapus job"})  
    return  
}  
  
if err := tx.Model(&models.Application{}).  
    Where("job_id = ?", job.ID).  
    Update("status", "dihapus").Error; err != nil {  
    tx.Rollback()  
    c.JSON(500, gin.H{"error": "Gagal update application"})  
    return  
}  
  
if err := tx.Where("job_id = ?", job.ID).  
    Delete(&models.Project{}).Error; err != nil {  
    tx.Rollback()  
    c.JSON(500, gin.H{"error": "Gagal hapus project"})  
    return  
}  
  
tx.Commit()  
c.JSON(200, gin.H{"message": "Job & semua application dihapus"})  
}
```

Penjelasan:

“Function DeleteJob ini digunakan untuk menghapus proyek milik client, tetapi dilakukan secara terkontrol menggunakan transaksi agar semua data yang berkaitan ikut terupdate dengan aman. Alurnya dimulai dengan mengambil id dari parameter URL, lalu sistem mengecek apakah user sudah login. Setelah itu, sistem mencari data job berdasarkan id dan memastikan bahwa yang mengakses adalah pemilik job tersebut. Jika bukan, maka akses akan ditolak.

Selanjutnya, sistem memulai transaksi database. Pertama, status job diubah menjadi ‘dihapus’ sebagai bentuk *soft delete*. Kemudian, semua data lamaran (application) yang terkait dengan job tersebut juga diupdate statusnya menjadi ‘dihapus’. Setelah itu, data project yang berhubungan dengan job tersebut benar-benar dihapus dari database.

Jika di salah satu proses terjadi error, maka semua perubahan akan dibatalkan menggunakan rollback agar data tetap konsisten. Jika semua proses berhasil, transaksi akan di-*commit* dan sistem akan mengembalikan response bahwa job beserta data terkait berhasil dihapus. Dengan cara ini, penghapusan menjadi lebih aman dan tidak menimbulkan inkonsistensi data.”

Controller – Mengelola Proyek – Update

```
func UpdateJob(c *gin.Context) {  
    id := c.Param("id")  
    userID, ok := getUserID(c)  
    if !ok {  
        c.JSON(401, gin.H{"error": "Unauthorized"})  
        return  
    }  
    var job models.Job  
    if err := config.DB.First(&job, id).Error; err != nil {  
        c.JSON(404, gin.H{"error": "Job tidak ditemukan"})  
        return  
    }  
  
    if job.ClientID != userID {  
        c.JSON(403, gin.H{"error": "Bukan job milikmu"})  
        return  
    }  
}
```

```

}

var input struct {
    Judul      string `json:"judul"`
    JobDesc    string `json:"job_desc"`
    KebutuhanProyek string `json:"kebutuhan_proyek"`
    KebutuhanSkill string `json:"kebutuhan_skill"`
    Budget     string `json:"budget"`
    LokasiPelaksanaan string `json:"lokasi_pelaksanaan"`
    Status     string `json:"status"`
}

if err := c.ShouldBindJSON(&input); err != nil {
    c.JSON(400, gin.H{"error": err.Error()})
    return
}

if job.Status == "dihapus" {
    c.JSON(400, gin.H{"error": "Job sudah dihapus"})
    return
}

updateData := map[string]interface{}{
    "judul":      input.Judul,
    "job_desc":   input.JobDesc,
    "kebutuhan_proyek": input.KebutuhanProyek,
    "kebutuhan_skill": input.KebutuhanSkill,
    "budget":     input.Budget,
    "lokasi_pelaksanaan": input.LokasiPelaksanaan,
    "status":     input.Status,
}

if err := config.DB.Model(&models.Job{}).
    Where("id = ?", id).

```

```
Updates(updateData).Error; err != nil {  
  
    c.JSON(500, gin.H{"error": "Gagal update job"})  
  
    return  
  
}  
  
c.JSON(200, gin.H{"message": "Job updated"})  
  
}
```

Penjelasan :

“Function UpdateJob ini digunakan untuk memperbarui data proyek yang dimiliki oleh client. Alurnya dimulai dengan mengambil id dari parameter URL, lalu sistem mengecek apakah user sudah login dengan mengambil userID. Setelah itu, sistem mencari data job berdasarkan id. Jika job tidak ditemukan, maka akan ditampilkan error. Selanjutnya dilakukan validasi untuk memastikan bahwa hanya client pemilik job yang dapat melakukan update. Jika bukan pemiliknya, maka akses akan ditolak.

Setelah validasi, sistem menerima data input dari request dalam bentuk JSON, seperti judul, deskripsi, kebutuhan proyek, skill, budget, lokasi, dan status. Jika input tidak valid, maka akan ditampilkan error. Kemudian sistem juga mengecek apakah job sudah berstatus ‘dihapus’. Jika iya, maka job tidak bisa diubah lagi.

Jika semua validasi sudah lolos, sistem akan menyusun data yang ingin diperbarui dalam bentuk map, lalu melakukan update ke database berdasarkan id job tersebut. Terakhir, sistem akan mengembalikan response bahwa data job berhasil diperbarui. Dengan fungsi ini, client dapat mengelola dan menyesuaikan informasi proyek sesuai kebutuhan.”